2026 CSP-S 复赛模考 2 题解

T1 截取密信

【题目大意】

给定两个仅由小写字母组成的字符串 s, t，其中 $|s| = n$、$|t| = m$ 且 $m \le n$。需要从 s 中选出一个严格递增的下标序列

$$p_1 < p_2 < \cdots < p_m,$$

使得 $s_{p_1} s_{p_2} \ldots s_{p_m}$ 与 t 至多有一个位置不同，并在所有可行方案中选出字典序最小的下标序列。无解时输出 -1。

【参考难度】

提高。

【题目解析】

为了让下标序列字典序最小，应该从左到右尽可能早地选择位置。但如果当前位置的字符与 t 当前字符不同，就必须确认：把这次不同作为唯一一次笔误后，后面是否还能全部匹配。

先从后往前预处理后缀匹配信息。令 `suf[i]` 表示从 s_i 开始向右，最多可以匹配 t 的多少个后缀字符。实际实现时，可以令 `j` 从 m 开始向前匹配：如果 $s_i = t_j$，就让 j 减一，最后得到当前位置之后还能匹配的长度。

从左到右扫描 s，当前需要匹配 t_j：

- 如果 $s_i = t_j$，直接选择下标 i；
- 如果 $s_i \neq t_j$，且还没有使用过笔误，并且选择 i 后剩余部分可以完全匹配，就把 i 作为唯一的错误位置；
- 否则跳过 i。

为了避免边界混乱，程序使用 1-based 下标，并在后缀数组中保留 `suf[n+1]` 表示空后缀。

为什么不能只找一个完全匹配的子序列?

因为题目要求的是字典序最小的方案，而允许一次错误有时可以让答案更小。例如：

```
s = babc, t = abc
```

完全匹配只能选择 $[2, 3, 4]$，但选择 $[1, 3, 4]$ 后得到 `bbc`，只在第一个位置不同，因此也是合法方案，并且

$$[1, 3, 4] < [2, 3, 4]。$$

所以即使数据保证存在完全匹配，也必须考虑使用一次修改。

测试点 $1 \sim 4$（20 分）：

$n, m \le 100$。枚举选出的下标序列，或枚举笔误位置后暴力匹配，复杂度可以控制在题目范围内。

一种直接做法是枚举 s 中选择的第一个位置，然后递归枚举后续位置，并统计已经出现的不同字符数。复杂度较高，但可以通过小数据。

测试点 $5 \sim 12$（60 分）：

$n, m \le 2000$。可以枚举笔误所在的下标，然后分别从左、右两侧贪心匹配：

- 左侧从 s 的前缀中匹配 t 的前缀；
- 固定笔误位置后，右侧从后往前匹配 t 的后缀；
- 只要左右两边能够拼成长度为 m 的答案，就用 `vector<int>` 保存当前下标序列，并直接比较字典序。

如果每次匹配都需要扫描字符串，复杂度为 $\mathcal{O}(nm)$，可以通过这一档数据。

特殊性质 A（测试点 $13 \sim 14$，10 分）：

性质 A 保证存在一个完全匹配的子序列，但仍然需要考虑一次笔误，因为一次笔误可能得到更小的字典序答案。使用后缀匹配数组后，可以在线性时间内完成贪心：

1. 正常匹配时优先选择当前位置；

2. 当前位置不匹配时，判断后缀是否还能完成匹配；
3. 可以完成则使用一次笔误，否则跳过当前位置。

复杂度为 $\mathcal{O}(n+m)$。

特殊性质 B（测试点 $15 \sim 16$，10 分）：

性质 B 保证不存在完全匹配的子序列，但不保证任意数据都有合法的“一处错误”方案。仍可以使用同样的线性贪心，只是最后需要检查是否确实选出了 m 个位置；无法完成时输出 -1。

正解（100 分）：

正解直接使用“后缀匹配 + 从左到右贪心”。不需要枚举笔误位置，也不需要 Z 函数。

设 `suf[i]` 表示从 s_i 开始，按照顺序最多能够匹配 t 的后缀起点。实现时从后往前扫描 s，维护当前还需要匹配的 t_j：如果 $s_i = t_j$，就令 j 减一，并记录 `suf[i]=j+1` 。

随后从左到右扫描 s，当前准备匹配 t_j：

- 若 $s_i = t_j$，直接选择 i；
- 若 $s_i \neq t_j$，且尚未使用过错误位置，同时 `suf[i+1]` 表示的后缀可以完成剩余匹配，则选择 i，并将这次选择作为唯一错误；
- 否则跳过 i。

判断条件的本质是：选择当前位置后，剩余部分必须能够全部正确匹配。由于我们已经从右到左预处理了后缀信息，所以可以在 $O(1)$ 时间完成判断。

为什么这种贪心能得到字典序最小的答案？扫描到位置 i 时，前面的选择已经固定。若选择 i 后仍然存在合法的后续方案，那么任何不选择 i 的方案都只能把当前下标换成更大的位置，因此字典序更大；只有在选择 i 会导致后缀无法完成时，才必须跳过 i。

时间复杂度为

$$\mathcal{O}(n+m),$$

空间复杂度为

$$\mathcal{O}(n+m)。$$

T2 谁能下到 100 层

【题目大意】

有 n 个从左到右排列的平台，第 i 个平台的高度为 h_i。可以从平台 i 跳到平台 j，当且仅当：

- $|i-j| \leq d$；
- $h_i > h_j$；
- i, j 之间的所有平台高度都严格小于 h_i。

可以从任意平台出发，求最多能经过多少个平台，出发的平台也计入答案。

【参考难度】

提高。

【题目解析】

直接按照题意从每个平台向左右扫描，遇到第一个高度不低于当前平台的位置就停止。由于每个位置最多要扫描 d 个位置，得到一个 $\mathcal{O}(nd)$ 的动态规划。

但正解需要进一步观察“不能被挡住”的条件。

固定平台 i，考虑它左侧第一个高度不小于 h_i 的平台。设这个平台为 p。那么从 i 向左跳时：

- p 之前的位置全部被 p 挡住，不能跳到；
- p 是左侧第一个可能成为跳跃终点的更高或等高平台；
- 如果 p 距离超过 d，则左侧不存在合法的跳跃终点；
- 如果 p 距离不超过 d，那么倒着跳时可以从 i 跳回 p。

注意，正着跳要求跳向更低平台；倒过来以后，边就变成从较低平台跳向能挡住它的最近更高平台。这样每个平台最多只有两条倒向边：左边最近更高平台和右边最近更高平台。

测试点 $1 \sim 4$（16 分）：

$n, d \leq 10$。可以使用记忆化搜索。

令 `dfs(i)` 表示从平台 i 出发最多能经过的平台数，初始为 1。向左、向右分别扫描至多 d 个位置：遇到高度不低于 h_i 的平台就停止，遇到更低的平台就转移到该平台。

如果记忆化，每个平台只会真正计算一次，每个状态最多扫描 $2d$ 个位置，复杂度为

$$\mathcal{O}(nd)。$$

没有记忆化时，相同状态会被重复计算，严格递增或严格递减数据上可能出现指数级重复调用，不能作为正式的记忆化搜索。

测试点 $5 \sim 12$（48 分）：

$n, d \leq 10^3$。直接使用上面的记忆化搜索即可，时间复杂度为 $\mathcal{O}(nd)$。

特殊性质 A（测试点 $13 \sim 15$，60 分）：

性质 A 为

$$h_1 > h_2 > \cdots > h_n。$$

此时没有平台会挡住从左向右或从右向左的扫描。若从平台 i 向右跳，则可以跳到右侧距离不超过 d 的任意平台；向左同理。

可以用滑动窗口维护转移范围中的最大 `dp` 值。按照高度或位置顺序处理后，每个状态只需从长度至多 d 的窗口中取最大值，复杂度可以降为 $\mathcal{O}(n)$ 或 $\mathcal{O}(nd)$，具体取决于实现。

特殊性质 B（测试点 $16 \sim 18$，72 分）：

性质 B 为 $d = n$，距离限制不再起作用。此时每个平台只需要找到左右两侧最近的更高平台。

对于每个位置 i，用单调栈求：

- `left[i]`：左侧最近的、满足 $h_{left[i]} > h_i$ 的平台；
- `right[i]`：右侧最近的、满足 $h_{right[i]} > h_i$ 的平台。

然后倒着进行 DP：

$$dp[i] = \max(dp[left[i]], dp[right[i]]) + 1。$$

正解（100 分）：

正解只需要在特殊性质 B 的单调栈方法上增加距离判断。这里采用纯 $\mathcal{O}(n)$ 的写法，不需要按高度排序。

从左到右扫描时维护一个单调递减栈。处理位置 i：

1. 弹出所有高度不大于 h_i 的位置；
2. 栈顶就是左侧第一个高度严格大于 h_i 的位置；
3. 如果 $i - S.top() \leq d$，则令 `left[i]=S.top()`，否则令其为不可达；
4. 将 i 入栈。

从右到左再做一次，得到 `right[i]`。

为什么只需要最近的更高平台？设左侧有两个更高平台 $p < q < i$，且 q 是离 i 最近的更高平台。平台 q 已经挡住了从 i 继续向左看向 p 的路径，因此 p 不可能成为合法的倒向跳跃终点。右侧完全相同。

得到两条边后，倒着跳的图是一个有向无环图，因为每条边都从较低平台指向较高平台。令 `dfs(i)` 表示从平台 i 倒着跳最多能经过的平台数，则：

$$dfs(i) = \begin{cases} 1, & \text{左右都没有可达的更高平台;} \\ \max(dfs(left[i]), dfs(right[i])) + 1, & \text{否则。} \end{cases}$$

使用记忆化数组后，每个平台只会计算一次，单调栈中的每个位置也只进栈、出栈一次，所以总复杂度为

$$\text{时间复杂度 } \mathcal{O}(n)，\qquad \text{空间复杂度 } \mathcal{O}(n)。$$

T3 站岗

【题目大意】

正方形营地边长为 a，边界上有 n 个候选位置，需要恰好选择 k 个位置，使所选位置两两之间的曼哈顿距离的最小值尽可能大。

【参考难度】

提高+/省选-。

【题目解析】

题目要求最大化“所有点对距离中的最小值”，典型地使用二分答案。

假设当前二分的答案为 D，问题变成：能否选出 k 个点，使任意两个点之间的曼哈顿距离都至少为 D？如果 D 可行，那么更小的距离也一定可行，因此可行性具有单调性。

关键是把正方形边界上的点映射到一个长度为 $4a$ 的环上。按逆时针方向定义周长坐标：

- 左边界 $x = 0$：映射为 $p = y$；
- 上边界 $y = a$：映射为 $p = a + x$；
- 右边界 $x = a$：映射为 $p = 3a - y$；
- 下边界 $y = 0$：映射为 $p = 4a - x$。

映射后，边界上的点按照顺时针顺序排列在环上。对于边界上的两个点，沿边界的两段距离分别为

$$\Delta = |p_i - p_j|, \qquad 4a - \Delta。$$

两点的曼哈顿距离恰好等于这两段弧长中的较小值：

$$\mathrm{dist}(i, j) = \min(\Delta, 4a - \Delta)。$$

因此，判定 D 是否可行，就是在环上选 k 个点，使相邻选点之间的环上距离（包括最后一个点回到第一个点）都至少为 D。

测试点 $1 \sim 2$（10 分）：

$n, k \le 15$。可以枚举所有 $\binom{n}{k}$ 种选法，计算每种选法中所有点对的曼哈顿距离，取最小值，再对所有方案取最大值。

复杂度约为

$$\mathcal{O}\left(\binom{n}{k} k^2\right)。$$

测试点 $3 \sim 6$（30 分）：

$n, k \le 1000$。对答案 D 二分。判定时枚举每个点作为环上的起点，然后从左到右贪心选择下一个距离至少为 D 的点。

固定起点后，选择下一个点时一定选当前能选的最靠左点。因为选择更靠左的位置不会减少后续可选位置，反而会给后续留下更多空间，所以这是最优的贪心。

如果已经选出 k 个点，再检查最后一个点与起点之间的环形距离是否至少为 D。枚举起点需要 $O(n)$，每次贪心需要 $O(n)$，因此一次 `check` 为 $O(n^2)$，总复杂度为

$$\mathcal{O}(n^2 \log a)。$$

测试点 $7 \sim 12$（60 分）：

此时 $k \le 25$。仍然二分答案，但判定时不再对每个起点重新扫描全部位置。

枚举第一个选中的点 `idx[0]`。对于后面的第 j 个点，维护指针 `idx[j]`，使其成为满足

$$p_{idx[j]} - p_{idx[j-1]} \ge D$$

的最靠左位置。随着起点右移，每个指针只会单调向右移动，因此固定一个答案后的总调整次数为 $O(nk)$。

一次判定复杂度为 $O(nk)$，二分答案后总复杂度为

$$\mathcal{O}(nk \log a)。$$

特殊性质 A（测试点 $13 \sim 14$，80 分）：

所有点在正方形的同一条边上。此时问题退化为在线段上选择 k 个点，使相邻点距离至少为 D。判定时从最左点开始贪心即可，不需要考虑环首尾相接的情况。

特殊性质 B（测试点 $15 \sim 16$，80 分）：

$k = n$，必须选择所有点。此时答案就是所有候选点两两曼哈顿距离的最小值。将点映射到环上后，只需要排序并检查相邻点之间的环形距离：

$$\min\left(p_{i+1} - p_i,\ 4a - p_n + p_1\right)。$$

正解（100 分）：

二分答案后，考虑固定某个起点。按照“选当前能选的最靠左点”的贪心，可以得到后续各个选点的位置。

直接为每个起点模拟会达到 $O(n^2)$。注意到：如果令每个点的父节点为从该点向右走至少 D 后能到达的第一个点，那么所有贪心选择构成一棵树：

- 根节点是哨兵，表示走出环后的位置；
- 每个真实点连接到它向右的第一个合法后继；
- 从根向下的一条路径，就对应一个固定起点下的贪心选点序列。

可以用双指针在线性时间内求出每个点的父节点。由于位置已经排序，父节点下标随着当前点变化具有单调性。

接下来使用 DFS 遍历这棵树，同时维护根到当前节点的路径。若当前路径上已经包含至少 k 个真实点，只需检查路径中第 k 个点与当前点之间的环形首尾距离是否至少为 D。路径上的第 k 个点可以用数组下标直接取得，不需要重新模拟整条路径。

一次 `check(D)` 的复杂度为 $O(n)$，二分答案的次数为 $O(\log a)$，因此：

$$\text{时间复杂度 } \mathcal{O}(n \log a),\qquad \text{空间复杂度 } \mathcal{O}(n)。$$

T4 种萝卜

【题目大意】

有一个 $n \times n$ 的网格。可以在每一列中选择一个深度 h_j，挖去该列第 $1 \sim h_j$ 行并种上一根萝卜，也可以不种植，记作 $h_j = 0$。

对于没有被挖去的格子 (i, j)，如果左侧或右侧相邻格至少有一个被挖去，则该格的养分会被吸收。求能吸收的最大总养分。

【参考难度】

省选/NOI-。

【题目解析】

以下统一使用 1-based 列号。令 B_j 表示第 j 列挖去的格子数，$0 \le B_j \le n$，边界外设 $B_0 = B_{n+1} = 0$。定义列前缀和

$$S_j[t] = \sum_{i=1}^{t} a_{i,j},\qquad S_j[0] = 0。$$

固定第 j 列及其左右相邻列后，能吸收到养分的行号满足

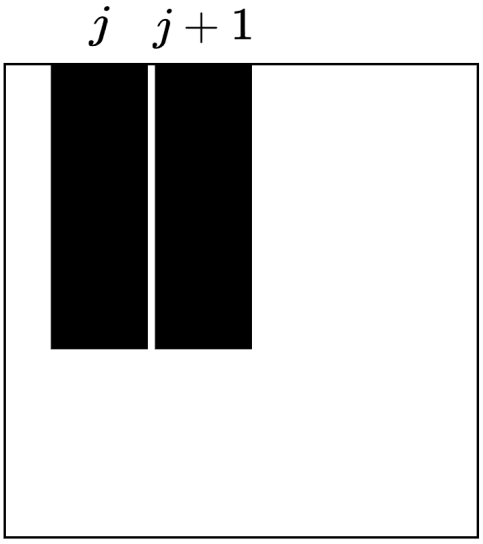
$$B_j < i \le \max(B_{j-1}, B_{j+1}),$$

因此第 j 列贡献为

$$G_j = \max\left(0, S_j[\max(B_{j-1}, B_{j+1})] - S_j[B_j]\right)。$$

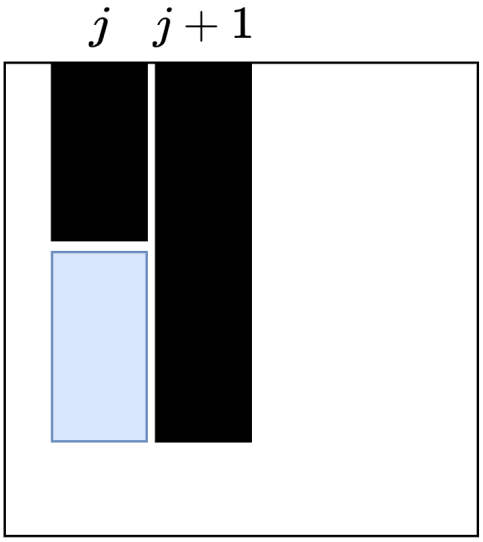
每列只结算一次，避免左右两侧同时影响时重复计分。

DP 状态



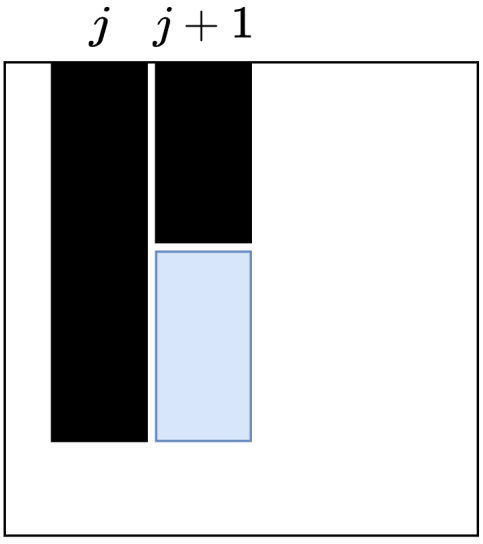
情况一:
第 j 列和第 $j+1$ 列的
黑格个数相等。

没有格子计入总分。



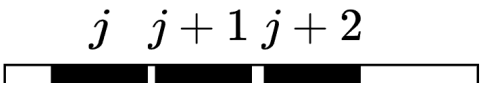
情况二:
第 $j+1$ 列的黑格个数更多。

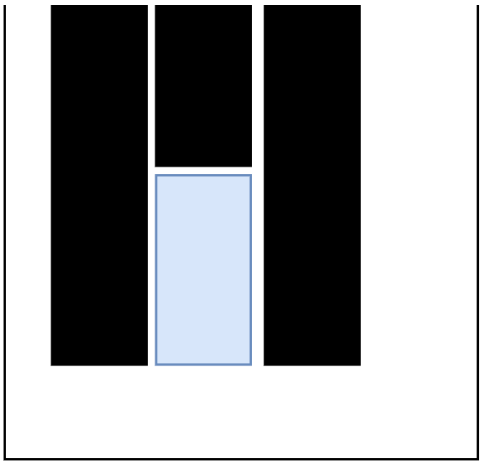
第 j 列的蓝色这段计入总分。



情况三（对吗？）:
第 j 列的黑格个数更多。

第 $j+1$ 列的蓝色这段计入总分。

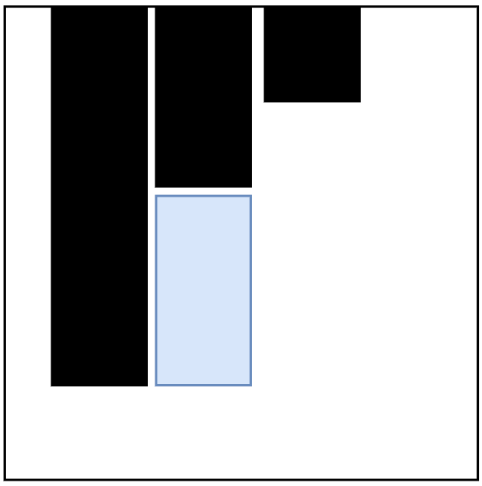




不对！如果出现这种情况，
蓝色部分岂不是会被计入两次?

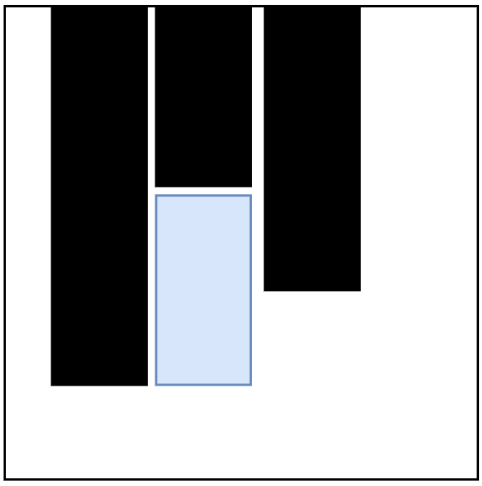
下面用 B 表示黑格个数。
把情况三细分为两种：
$B_j > B_{j+1} \geq B_{j+2}$。
$B_j > B_{j+1} < B_{j+2}$。

$j \quad j+1 \; j+2$



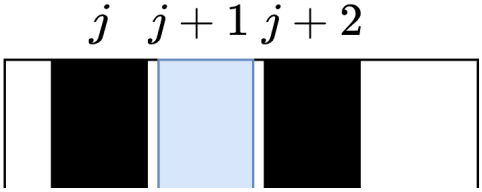
情况三：
只有在 $B_j > B_{j+1} \geq B_{j+2}$ 时，
才把第 $j+1$ 列的蓝色这段计入总分。

$j \quad j+1 \; j+2$

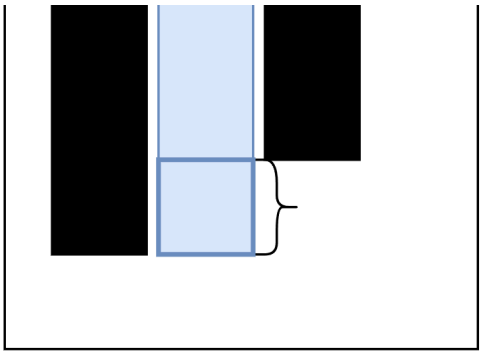


情况四-1：
$B_j > B_{j+1} < B_{j+2}$
这种情况怎么办?

$j \quad j+1 \; j+2$



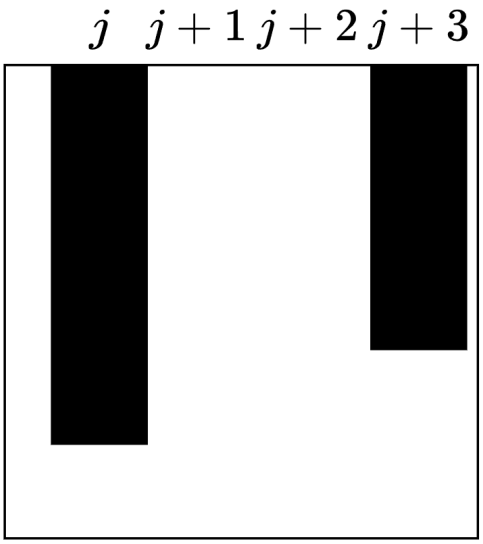
情况四-2：
贪心，由于左右都是黑格，
第 $j+1$ 列全白更好。



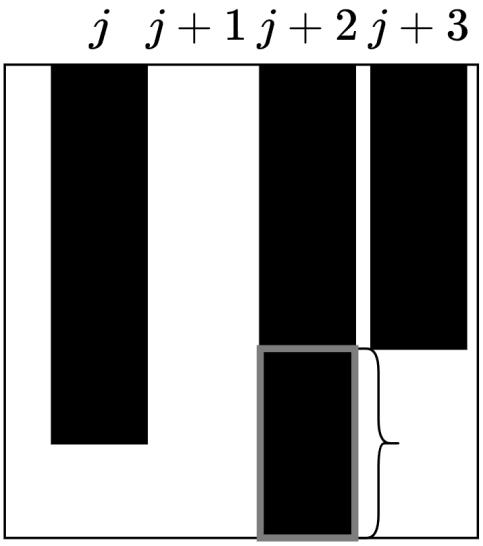
但仍然没有解决重复统计的问题。

人为规定不在 j 统计这种情况?
如果 $B_j \le B_{j+2}$，这样规定没有问题。
但如果 $B_j > B_{j+2}$，
多出来的这一段蓝色岂不是漏掉了?

在下面的讨论中，$B_j > B_{j+2}$。



情况四-3：
引入第 $j+3$ 列。
如果 $B_{j+2} \le B_{j+3}$，
结合 $B_j > B_{j+2}$，
第 $j+2$ 列涂黑是没有意义的，
所以 $B_{j+2} = 0$ 是最优的。
但这不属于情况四，因为不满足
$B_j > B_{j+1} < B_{j+2}$。



情况四-4：
如果 $B_{j+2} > B_{j+3}$，
由于超出 B_{j+3} 的那段左右全是白色，
所以 $B_{j+2} = n$ 是最优的。
注意这种情况不在乎 B_j 是多少。

所以不存在 $B_j > B_{j+2}$ 的情况。
因此，为避免重复统计，可以人为规定：
$B_j > B_{j+1} < B_{j+2}$ 时不计入。

综上所述，只有两种情况，才会计入总分：
1. $B_j < B_{j+1}$，计入第 j 列的格子。
2. $B_j > B_{j+1} \ge B_{j+2}$，计入第 $j+1$ 列的格子。

图中将相邻列深度关系归纳为两类。对每个 DP 层和边界深度 p，分别维护两类转移对应的最大已计分值，记为 $f_j(p,0)$、$f_j(p,1)$，其中 $0 \le p \le n$。状态 1 保存第一类（前缀）转移所得的最优值；状态 0 除了包含同一批候选值，还会并入第二类（后缀）转移的候选值。标记只区分关系类别，不代表另一列的具体深度。初始化时还没有计入任何列的得分，因此

$$f_1(p,0) = f_1(p,1) = 0 \qquad (0 \le p \le n)。$$

对 $j = 1, 2, \ldots, n-1$，用第 j 层转移到第 $j+1$ 层。处理完所有列后，答案为

$$\max_{0 \le p \le n} f_n(p,0)。$$

状态转移

先看 $p > 0$ 的第一类转移。若上一层的深度为 $q < p$，第 j 列新增的得分为 $S_j[p] - S_j[q]$；另一种情况是不在这一步增加该段得分。因此第一类转移的最大值为

$$\max\left(f_j(p,0),\ S_j[p] + \max_{0 \le q < p}\{f_j(q,1) - S_j[q]\}\right)。$$

这同时作为新一层两类状态的初值：

$$f_{j+1}(p,1) = \max\left(f_j(p,0),\ S_j[p] + \max_{0 \le q < p}\{f_j(q,1) - S_j[q]\}\right)。$$

第 $j+1$ 层的状态 0 至少可以取这个候选值。再考虑 $q > p$ 的第二类转移：此时第 $j+1$ 列新增的得分为 $S_{j+1}[q] - S_{j+1}[p]$，所以

$$f_{j+1}(p,0) = \max\left(f_{j+1}(p,0),\ -S_{j+1}[p] + \max_{p < q \le n}\{f_j(q,0) + S_{j+1}[q]\}\right)。$$

边界深度 $p = 0$ 单独处理：

$$f_{j+1}(0,0) = \max_{1 \le q \le n}\{f_j(q,0) + S_{j+1}[q]\},$$

$$f_{j+1}(0,1) = \max\left(f_j(0,0), f_j(n,0)\right), \qquad f_{j+1}(0,0) = f_{j+1}(0,1)。$$

两类边界状态取相同值：此时 $p = 0$，由边界处的转移得到的候选可用于两类状态。第一式不考虑 $q = 0$，因为连续三列都未挖时不会产生新增得分。若直接枚举每个 p 对应的所有 q，每层需要 $O(n^2)$，总时间复杂度为 $O(n^3)$。

前后缀最大值优化

按 p 从小到大计算第一类转移时，维护

$$\max_{0 \le q < p}\{f_j(q,1) - S_j[q]\}$$

即可在线性时间内算出所有 p 的结果。按 p 从大到小计算第二类转移时，维护

$$\max_{p < q \le n}\{f_j(q,0) + S_{j+1}[q]\}$$

再减去 $S_{j+1}[p]$。边界 $p = 0$ 的转移只需额外扫描一次。因此每层为 $O(n)$，总时间复杂度为 $O(n^2)$；前缀和与 DP 的空间复杂度均为 $O(n^2)$。

特殊性质

特殊性质 A：所有格子的养分值均为 c。 对任意一行，令 k 为挖去格子数，q 为能吸收养分的未挖格子数。每个挖去格子至多相邻两个得分格，且两类格子不重叠，所以 $q \le 2k$、$q + k \le n$，进而 $q \le \lfloor 2n/3 \rfloor$。按“黑、白、白”重复排列可以达到上界，答案为

$$c \times n \times \left\lfloor \frac{2n}{3} \right\rfloor。$$

特殊性质 B：每列内部养分值相同。 设第 j 列每格养分为 c_j。把每一行单独看成黑白列状态，就是一个带权的一维选择问题。任意方案的每一行得分都不超过该问题的一行最优值；把一行最优状态在所有行重复即可达到上界。因此只需求出一行最优值，再乘 n；一行的 DP 只需记录相邻两列的黑白状态，时间复杂度为 $O(n)$。